

Automate Delivery Follow-up and Connect Work Systems

Session 25

Product and Delivery Teams

Start from a workflow people already understand

Sessions 20-24 created the brief, issues, canvas, and reviewed work.

Now automate one recurring follow-up.

Start with the preflight

Check	Required answer
GitHub Copilot	Can each learner open Automations now?
GitHub MCP	Can the automation read the approved issues?
Policy	Is this surface and feature approved here?
Data	Which inputs may enter the workflow?
Meter	Which usage and cost signals will we watch?
Owner	Who decides whether the result is usable?
Stop guard	What ends the run or disables the workflow?
Recovery procedure	How will we stop, preserve evidence, and return control to the owner?

No Copilot access means stop. Resolve access before the lab.

Autonomy levels

Level	Allowed behavior	Approval
A0	Checklist or template	Human performs the work
A1	Draft an output	Human approves every use
A2	Take a bounded, reversible action	Contract approval plus exception review
A3	Act without per - run approval	Monitoring and immediate stop path

Use the lowest level that still saves useful time.

Create one draft-only automation

- Retrieve approved issue state.
- Read the synthetic delivery inputs.
- Draft the weekly update.
- Return run evidence.
- Do not publish or change work items.

Service Request Portal

Every Friday, prepare an internal status draft from:

- synthetic service request records;
- a short change log;
- the approved contract.

The lab workflow is **A1: Draft**. It cannot publish or change source records.

Review the run in order

1. Confirm the contract and run identity.
2. Check inputs, tools, and data boundaries.
3. Reconcile source, accepted, and rejected counts.
4. Trace each claim to evidence.
5. Confirm that no prohibited side effect occurred.
6. Record the Copilot surface and available usage evidence.

Exceptions are evidence

One source record has no owner. The run must exclude it, explain why, and name the escalation path.

Silently repairing or dropping a record fails the contract.

Decide

Decision	Use when
Accept	Evidence is complete and the output meets the contract
Reject	The result is wrong, unsupported, incomplete, or out of bounds
Pause	Policy, data, meter, owner, or evidence is unresolved

Record the reason, reviewer, time, and next action.

Measure useful time saved

```
manual baseline
- run time
- review time
- rework
- allocated overhead
= useful time saved
```

The reference run: `42 - 8 - 12 - 4 - 3 = 15 minutes`

Stakeholder-ready update

State:

- what period and workflow the update covers;
- what changed and which exception remains;
- what the reviewer accepted;
- what happens next and who owns it.

Do not expose prompts, internal reasoning, secrets, or unsupported claims.

Choose the authoritative work system

Keep **GitHub** authoritative when issues and agent work live there.

Keep **Azure Boards** authoritative when the delivery backlog lives there.

Do not maintain competing status fields without an owner.

Product facts can change

GitHub documents local and cloud automations, supported triggers, tool controls, policy dependencies, visibility, and usage metering.

Check the current official documentation before delivery. Do not promise access or rely on a fixed interface.

Lab deliverable

1. Automation contract
2. GitHub Copilot-reviewed evidence packet
3. Approval record
4. Stakeholder-ready update

Run one live draft-only automation. Review its evidence, then choose **keep**, **revise**, or **disable**.